

Salesforce Compass Program — Full Project Breakdown

Team Setup

- **You:** Backend, ML/AI integrations, Python/FastAPI, Node.js, Cloud SDKs
 - **Teammate:** Frontend (React/Next.js)
 - **Duration:** 8 weeks, ~2-3 hrs/day minimum
 - **Stack comfort:** Python/FastAPI/Flask, Node.js/Express, ML/AI integrations
-

PROBLEM STATEMENT 1

Low-Code/No-Code Security Layer with Visual RBAC Builder

What it is

A drag-and-drop visual builder that lets non-developers design Role-Based Access Control (RBAC) policies. Users place UI blocks, define roles, attach rules, and the system generates enforceable API middleware behind the scenes.

Core Architecture



Components Breakdown

Layer	Tech	Owner
Visual block builder UI	React + React Flow	Teammate
Policy JSON schema	Custom DSL / JSON Schema	You
Policy Engine API	FastAPI	You
Rule evaluation logic	Python (Rego-inspired)	You
Auth (JWT/OAuth2)	FastAPI + python-jose	You
Permission store	MongoDB or PostgreSQL	You
API Gateway middleware	Express.js or FastAPI middleware	You
Dashboard/analytics	React + Recharts	Teammate
Deployment	Docker + Render or Railway	You

What Makes This Stand Out (Your ML Angle)

This is where you go beyond basic RBAC and make it major:

1. Anomaly Detection on Access Patterns
- Train a simple Isolation Forest or Autoencoder on access logs
 - Flag unusual API call patterns (e.g., an "iPhone discount giver" role calling payment APIs at 3am)
 - Libraries: `scikit-learn`, `PyOD`
2. AI-Assisted Rule Suggestion
- When a user creates a new role, call Claude/OpenAI API to suggest sensible rules based on role name and context
 - Example: User types "Customer Support Agent" → system suggests: read on tickets, write on notes, deny on billing
 - This is a killer feature for a no-code tool
3. Policy Conflict Detector
- ML + rule engine that detects when two policies contradict each other
 - E.g., Role A gives write access, but Rule B (higher priority) denies write — highlight this in the UI
4. Natural Language to Policy
- Text box where user types: "Managers can approve discounts up to 20% but only during business hours"

- NLP layer (spaCy or LLM prompt) converts this to structured policy JSON
- This is your **headline feature**

8-Week Schedule — PS1

Week 1 — Foundation

You	Teammate
Design Policy JSON schema / DSL	Setup Next.js + Tailwind + shadcn/ui
Set up FastAPI project, Docker	Create wireframes in Figma or Excalidraw
Design MongoDB schema (users, roles, rules, policies)	Build basic app shell: sidebar, top nav, canvas area
JWT auth flow (register/login endpoints)	Auth pages: login, register

Week 2 — Core Engine

You	Teammate
Build Policy Engine: evaluate(user, resource, action)	Build drag-and-drop canvas using React Flow
Role CRUD endpoints	UI blocks: Role block, Rule block, Condition block
Rule CRUD endpoints	Connect blocks via edges — visual graph
Write unit tests for policy eval	Export policy button (dumps canvas state to JSON)

Week 3 — API Gateway Middleware

You	Teammate
Build Express.js middleware that calls Policy Engine	Role management panel (list, create, edit roles)
Protect sample routes (admin, editor, viewer)	Rule assignment UI
API call flow: Request → middleware → policy check → allow/deny	Visual feedback: green/red access indicators
Logging layer: log every access decision to DB	User management page

Week 4 — ML Feature 1: Anomaly Detection

You	Teammate
Build access log collector (Celery or background task)	Dashboard: access logs table, filterable
Train Isolation Forest on synthetic log data	Charts: API call heatmap, role usage stats
Expose <code>/anomalies</code> endpoint	Alert banner in UI when anomaly flagged
Write anomaly score to log records	Notification system (toast/banner)

Week 5 — ML Feature 2: NLP Policy Builder

You	Teammate
Build NLP endpoint: text → structured policy JSON	"Natural Language" input box in UI
Prompt engineering for LLM-based policy extraction	Preview panel: shows parsed policy before applying
Policy conflict detection logic	Conflict warning UI: highlights conflicting blocks
Integration test: full flow end to end	Polish block editor, add undo/redo

Week 6 — AI Rule Suggester + Refinements

You	Teammate
Build AI suggestion endpoint for roles	Auto-suggestion dropdown when naming a role
Role template library (prebuilt: Admin, Editor, Viewer, Support)	Template picker modal
Policy versioning: save/load policy snapshots	Version history panel
Performance: cache policy decisions (Redis)	Loading states, skeleton screens

Week 7 — Integration + Demo App

You	Teammate
Build a demo "E-Commerce" app with 5 protected routes	Style the full app: consistent design system
Wire the demo app through the policy middleware	Mobile-responsive layout
End-to-end testing (pytest + Playwright)	User onboarding flow (guided tour)
Write API documentation (Swagger auto-gen)	Landing/marketing page for the project

Week 8 — Polish, Deployment, Presentation

You	Teammate
Deploy on Render/Railway + Mongo Atlas	Final UI polish
Set up monitoring: uptime, error rate	Record demo video
Write README + architecture diagram	Prepare slide deck
Security hardening: rate limiting, input validation	Accessibility pass

Technical Stack — PS1

Backend (You)

- Python 3.11, FastAPI, Pydantic v2
- MongoDB + Motor (async driver) or PostgreSQL + SQLAlchemy
- Redis (caching policy decisions)
- Celery (background log processing)
- scikit-learn, PyOD (anomaly detection)
- spaCy or OpenAI/Anthropic API (NLP policy builder)
- python-jose (JWT), passlib (bcrypt)
- pytest, httpx (testing)

Frontend (Teammate)

- Next.js 14 (App Router)
- React Flow (drag-and-drop canvas)
- shadcn/ui + Tailwind CSS
- Recharts (analytics)
- Zustand or React Query

Infrastructure

- Docker + docker-compose
- Render or Railway (free tier OK)
- MongoDB Atlas (free tier)
- GitHub Actions (CI/CD)

PROBLEM STATEMENT 2

GhostPath: Fragmented Multi-Cloud Storage Vault

What it is

A privacy-first file storage system that shreds files into encrypted fragments and distributes them across AWS S3, GCP Cloud Storage, and Azure Blob simultaneously. No single provider holds a complete or readable file. Uses Shamir's Secret Sharing (threshold cryptography) so reconstruction requires k-of-n fragments.

Core Architecture



Components Breakdown

Layer	Tech	Owner
File upload/download UI	React + Next.js	Teammate
Vault dashboard	React + Recharts	Teammate
Fragment API server	Node.js + Express	You
Fragmentation/reconstruction engine	Python (secretsharing lib)	You
AES-256 encryption layer	Python (cryptography lib)	You
AWS S3 SDK	boto3 (Python)	You
GCP Cloud Storage SDK	google-cloud-storage	You
Azure Blob SDK	azure-storage-blob	You
Metadata store	MongoDB	You
Self-healing monitor	Python + Celery	You
Key management	Python (custom KMS or HashiCorp Vault)	You

What Makes This Stand Out (Your ML + Engineering Angle)

1. Intelligent Fragment Placement

- Instead of round-robin placement, train a simple model on provider latency + uptime history
- Prefer faster providers for hot files, cheaper providers for cold storage
- Libraries: scikit-learn (simple regression), or even just heuristics with real telemetry

2. Self-Healing Mechanism (the "killer feature" from the PS)

- Background Celery beat job checks fragment availability every N minutes
- If a provider returns 404 or times out: reconstruct from remaining fragments → re-replicate to backup provider
- This is genuinely impressive engineering — automated resilience

3. Deduplication Before Encryption

- Hash file content before encrypting (content-addressable storage)
- If two users upload the same file, store fragments once, save storage costs
- Use SHA-256 of plaintext before encryption as the content hash

4. Access Pattern Analytics

- ML clustering on access logs to classify files as "hot", "warm", "cold"

- Automatically migrate cold files to cheaper storage tiers
- Show this in the dashboard as a "storage intelligence" feature

5. Threat Simulation Mode

- Demo feature: simulate a cloud provider going down
- Show the self-healing kick in live in the UI — this is a jaw-dropping demo moment

8-Week Schedule — PS2

Week 1 — Foundation

You	Teammate
Setup Python fragmentation service (FastAPI or Flask)	Setup Next.js + Tailwind + shadcn/ui
Install and test cloud SDKs (boto3, GCP, Azure)	File upload component with drag-and-drop
Design MongoDB schema: files, fragments, providers	Wireframes: vault dashboard, upload modal
Implement AES-256 encrypt/decrypt using <code>cryptography</code> lib	Auth pages + basic layout

Week 2 — Shamir's Secret Sharing

You	Teammate
Implement SSS using <code>secretsharing</code> or <code>tss</code> Python lib	File listing page (name, size, status, date)
Test: split file into 5 fragments, recover from any 3	File detail panel: fragment map visualization
Fragment upload to all 3 cloud providers	Progress bar during upload
Metadata write to MongoDB on successful distribution	Vault stats: total files, storage used per provider

Week 3 — Reconstruction + Download

You	Teammate
Fragment fetch from all 3 providers	Download button with status states
Reconstruct file from k-of-n fragments	Fragment health indicators (green/red per provider)
Decrypt and stream file back to user	"Provider Map" component: shows which provider holds what
Handle partial availability (1 provider down)	Error states: graceful degradation UI

Week 4 — Self-Healing Engine

You	Teammate
Celery beat job: health check all fragment locations	Real-time healing progress in UI (WebSocket or polling)
Detect missing/corrupted fragments	"Threat Simulator" toggle in UI
Trigger re-replication from surviving fragments	Alert system: notify user when healing triggered
Test: manually delete S3 fragment, watch healing kick in	Healing log table: timestamps, actions taken

Week 5 — ML Feature: Intelligent Placement + Tiering

You	Teammate
Latency/uptime telemetry collection per provider	Storage analytics dashboard
Fragment placement scorer (heuristic + simple model)	File classification badges: hot/warm/cold
Content-addressable storage (dedup by SHA-256 hash)	Dedup indicator on file cards
Access frequency tracker (Redis counter per file)	File access history chart

Week 6 — Key Management + Security Layer

You	Teammate
Per-user encryption key management (PBKDF2 + envelope encryption)	Security settings page
Zero-knowledge design: server never holds plaintext keys	Key info panel (show key fingerprint, never raw key)
Implement file sharing: generate time-limited decryption tokens	Share button + link expiry UI
Audit log: every access event recorded	Audit log page with filters

Week 7 — Demo Polish + Integration

You	Teammate
Full end-to-end integration tests	Demo mode: guided walkthrough
Performance: chunked upload for large files	Upload progress with chunk indicator
Stress test: 50 concurrent uploads	Responsive layout, mobile polish
Write detailed architecture README	Landing page / project site

Week 8 — Deploy + Present

You	Teammate
Deploy: Vercel (frontend) + Render (API) + MongoDB Atlas	Final UI polish
Wire real AWS S3, GCP, Azure free tier accounts	Record demo showing threat simulation
Monitoring: uptime checks on all 3 providers	Prepare slide deck
Security audit: check for key leakage in logs/responses	Practice demo presentation

Technical Stack — PS2

Backend (You)

- Python 3.11, FastAPI
- `cryptography` (AES-256-GCM)
- `secretsharing` or `tss` (Shamir's Secret Sharing)

- boto3 (AWS S3)
- google-cloud-storage (GCP)
- azure-storage-blob (Azure)
- Celery + Redis (self-healing scheduler)
- MongoDB + Motor
- scikit-learn (placement scoring)
- hashlib (SHA-256 dedup)

Frontend (Teammate)

- Next.js 14
- shadcn/ui + Tailwind
- Recharts or Chart.js
- Framer Motion (animations)
- Socket.io client (real-time healing updates)

Infrastructure

- Vercel (frontend)
- Render (Python API)
- MongoDB Atlas
- AWS S3 free tier
- GCP Cloud Storage free tier (5GB)
- Azure Blob free tier
- GitHub Actions

HEAD-TO-HEAD COMPARISON

Complexity vs. Reward

Dimension	PS1 (RBAC Builder)	PS2 (GhostPath)
ML Depth	★★★★☆ (anomaly detection, NLP)	★★★☆☆ (placement, tiering)
Engineering Depth	★★★☆☆ (middleware, policy engine)	★★★★★ (crypto, multi-cloud, SSS)
Real-world applicability	★★★★★ (every SaaS needs RBAC)	★★★★☆ (niche but growing)
Demo wow factor	★★★☆☆	★★★★★ (threat sim is electric)

Dimension	PS1 (RBAC Builder)	PS2 (GhostPath)
Teammate workload	Balanced	Slightly frontend-heavy mid-project
Your backend workload	Balanced	Heavy (3 cloud SDKs + crypto)
Risk level	Low	Medium-High (crypto complexity)
Salesforce relevance	Very high (identity, access mgmt)	Moderate
Uniqueness	Medium (RBAC is well-known)	High (SSS + multi-cloud is rare)

Honest Recommendation

Choose PS1 if:

- You want Salesforce-specific relevance (they care deeply about identity and access management)
- You want the ML NLP feature to be the centrepiece
- You want more predictable scope and lower risk
- You're aiming for a production-ready, polished product

Choose PS2 if:

- You want to build something genuinely rare that interviewers/judges haven't seen before
- You're comfortable with cryptography fundamentals
- The "threat simulation demo" excites you — this is the type of live demo that wins competitions
- You want to show multi-cloud engineering depth

Overall verdict: PS2 (GhostPath) has the higher ceiling. PS1 has the safer floor.

ADDITIONS YOU SHOULD BUILD REGARDLESS OF CHOICE

These apply to both and will make either project exceptional:

1. **A real metrics dashboard** — not just the feature, but telemetry on your own system (latency, error rates, throughput). Use Prometheus + Grafana or just a custom dashboard.
2. **A polished public demo** — don't just show it locally. Deploy it. Have a URL you can share.
3. **Architecture diagram** — a clean, professional diagram showing all components. Judges love this.
4. **Load testing results** — run Locust or k6, show what load your system handles. Numbers impress.
5. **A "What I would build next" slide** — shows maturity and foresight.

